

# 长上下文场景中 LLM 自动数据标注挑战赛技术报告

团队名称：橄榄鲨

对应预测包：[final\\_submit\\_20260520\\_task2\\_noun\\_highconf\\_on\\_8160.zip](#)

预测包 SHA256：[ec4d57809664d4ba084c374be716f24d579951c4376a5c94232afaff8f436029](#)

报告日期：2026 年 5 月 21 日

## 摘要

本方案面向“长上下文场景中 LLM 自动数据标注挑战赛”，在只使用官方数据集、Qwen3-4B 模型和 FlagScale 推理框架的约束下，设计了一套可复现的长上下文 ICL 自动标注流程。整体思路不是简单堆叠更多 few-shot 样例，而是把长上下文拆成“规则理解、示例结构化、任务类型路由、模型复核、格式抽取和执行校验”几个可解释环节。对于任务 1-7，方案重点控制模型在长上下文中的注意力漂移和答案粒度；对于任务 8，方案将代码生成改造成“模型生成候选代码 + 公开语义执行检查 + 通用 wrapper 修复”的闭环。

最终预测包对应 FlagOS 平台已评测通过的 [81.72](#) 线上最高分方案。为保证开源材料与排行榜成绩一致，本报告、默认复现脚本和代码包中的 [final\\_prediction.zip](#) 均指向该已验证版本。Task7 copy-consensus 复核作为后续实验分支保留在代码中，但默认不开启，也不计入本报告最终成绩。

## 1. 赛题要求理解

### 1.1 官方约束

根据赛事页面和 OpenSeek 官方仓库说明，本赛题要求参赛方案满足以下核心约束：

| 项目   | 要求                                  | 本方案处理                                                                           |
|------|-------------------------------------|---------------------------------------------------------------------------------|
| 模型   | 必须使用 Qwen3-4B                       | 最终复现链路面向 Qwen3-4B，实验过程中的其它接口仅作开发加速，不进入最终复现口径                                    |
| 框架   | 必须使用 FlagScale 作为模型加载和运行底层框架        | 官服复现路径保存 FlagScale 配置、启动脚本和调用记录                                                 |
| 数据   | 仅允许使用组委会官方数据集                       | 不使用外部知识库、外部标注数据或自建训练集                                                           |
| 任务   | 8 个数据集均需提交预测                        | 预测包包含 <a href="#">openseek-1-v1.jsonl</a> 至 <a href="#">openseek-8-v1.jsonl</a> |
| 长上下文 | Task1-7 最小 ICL 上下文约 30K，Task8 约 16K | 构造 Round 1 长上下文规则理解轮，记录 token 数、示例 ID 和 prompt hash                             |
| 提交格式 | 8 个 jsonl 直接打包为 zip                 | 使用本地 validator 检查文件名、行数、字段和空预测                                                  |

OpenSeek 数据说明中列出 Task1-7 的最小 ICL 上下文为 30K，Task8 为 16K；每个任务均包含 [Definition](#)、[examples](#)、[test\\_samples](#)，其中测试样本不含标签。输出说明要求每个 jsonl 行包含 [test\\_sample\\_id](#) 和 [prediction](#) 两个字段，zip 内直接包含 8 个任务文件。

### 1.2 赛题的三个关键问题

本方案围绕赛题提出的三个关键问题展开：

第一，提示策略设计。长上下文下模型容易受无关样例影响，出现格式漂移、答案粒度漂移或直接复制题面词的问题。因此本方案固定“任务定义、标签规则、长上下文示例、当前输入、输出约束”的模块顺序，并在不同任务上使用结构化输出模板。例如计数

任务要求输出整数，Task1 要求输出可检查 JSON，Task8 要求输出可执行代码。

第二，超长上下文构造。官方 examples 数量远超单次最优 few-shot 数量，直接全量拼接会让 Qwen3-4B 被噪声稀释。我们采用“长上下文规则理解轮 + 当前样本推理轮”的两轮设计。Round 1 将任务定义、覆盖性示例和当前样本相关示例组织成可追踪的长上下文输入，让模型先归纳标签边界、输出粒度和易错点；Round 2 再使用该轮沉淀的任务理解进行稳定输出。

第三，多轮对话与持续标注一致性。我们不让多轮对话无限展开，而是把多轮限制为可追踪的复核机制：主答案通道、结构化复核通道、候选一致性门控和格式抽取。Python 只负责解析、比较和打包，不根据隐藏标签直接生成答案。

## 2. 总体方案

### 2.1 流程概览

整体流程如下：

官方数据集

-> 任务定义解析与样本画像

-> 示例检索、聚类、重排

-> 长上下文规则理解 Round 1

-> 任务类型路由与短提示 Round 2

-> Qwen3-4B / FlagScale 推理

-> 模型复核、候选一致性门控

-> 格式抽取与执行型校验

-> 8 个 jsonl + zip 提交包

方案中每一步都保留可复现记录。长上下文构造记录 `selected_example_ids`、`round1_prompt_sha256`、`round2_prompt_sha256`、`input_sha256` 和 token 统计；提交包构建记录基线 zip、改动数量、改动原因和最终 SHA256。

### 2.2 长上下文构造

我们将每个任务的 examples 分为两类：

| 示例类型              | 作用                 | 排布位置               |
|-------------------|--------------------|--------------------|
| Coverage examples | 覆盖任务规则、标签空间和常见边界情形 | prompt 中部，按类型聚类    |
| Core examples     | 与当前样本相似，直接帮助推理     | 当前输入前部或尾部，靠近模型输出位置 |

排序原则是“先稳定规则，再提供覆盖，最后给当前样本相关证据”。这样做的原因是，长上下文模型虽然能读取大量 token，但并不等于能从任意位置稳定抽取关键规则；把最相关示例放在当前样本附近，可以减少“中间遗忘”和相似样例干扰。

当前证据链在每个任务抽样 3 条样本检查：Task1-7 的 Round 1 prompt 最小 token 数均超过 30K，Task8 最小 token 数超过 16K。统计如下：

| 任务    | 最小 Round 1 token | 最大 Round 1 token | 平均选中示例数 |
|-------|------------------|------------------|---------|
| Task1 | 30185            | 30198            | 381.67  |
| Task2 | 30138            | 30184            | 409.33  |
| Task3 | 30135            | 30217            | 323.67  |
| Task4 | 30135            | 30212            | 340.67  |
| Task5 | 30137            | 30195            | 393.00  |
| Task6 | 30170            | 30200            | 324.67  |
| Task7 | 30147            | 30223            | 345.33  |
| Task8 | 16353            | 17043            | 13.00   |

### 2.3 模型调用与合规边界

最终复现路径使用 FlagScale 启动 Qwen3-4B 服务，并通过 OpenAI-compatible API 调用。根据官方 README，模型需要下载 Qwen3-4B 权重，并在配置中设置 YaRN rope scaling：

```
"rope_scaling": {
  "rope_type": "yarn",
  "factor": 4.0,
  "original_max_position_embeddings": 32768
}
```

本地快速实验阶段使用过 LM Studio 接口加速消融，但技术报告和开源复现不依赖 LM Studio。最终代码保留 FlagScale 配置、启动脚本和长 prompt 调用日志，确保评审阶段可以在官方环境复现。

## 3. 分任务设计

### 3.1 Task1: closest integers

Task1 要求找出整数列表中任意两数的最小绝对差。早期 few-shot 方式容易出现两个问题：一是模型漏看重复值，二是模型在长上下文示例干扰下把示例数字混入当前题。最终方案把提示改成结构化计算过程：要求模型输出 `target_input`、`sorted`、`adjacent_pairs` 和 `min_diff`，并在提示中强调 `sorted` 必须只包含当前输入数字。

Python 后处理只做 JSON 提取、字段完整性检查和最终 `min_diff` 抽取。若模型输出结构不完整，则触发同一模型的自检提示重新输出。这样做既保持答案来自 Qwen3-4B，又避免把简单格式错误扩大成错题。

### 3.2 Task2: count nouns / verbs

Task2 是名词和动词计数任务，难点不在输出格式，而在词性边界。Qwen3-4B 对动名词、专有名词、复合名词、助动词和短语动词的判断不稳定。我们为 Task2 设计了“结构化任务规范 + 词性边界样例 + 高置信复核”的流程。

具体做法是：

- 1. 先根据题目要求判断当前样本要数 noun 还是 verb；
- 2. 使用固定规则提示明确不计入的边界，如代词、形容词、冠词、介词；
- 3. 对 noun 类高风险样本，调用 Qwen3-4B 复核，要求列出候选词和最终计数；
- 4. 只有当复核值与当前答案差异很小、且多个提示通道一致时才接受修改。

最终高置信 noun 分支修改 77 条样本，线上包 `final_submit_20260520_task2_noun_highconf_on_8160.zip` 得到 **81.72**，成为本报告最终组合包的基线。

3.3 Task3: Collatz sequence

Task3 要求生成 Collatz 序列。该任务具有明确算法规则，但模型直接长文本生成时容易漏项、提前停止或输出解释。方案把提示限制为“只输出 JSON list”，并在模型内部显式描述规则：偶数除以 2，奇数乘 3 加 1，直到 1。

在合规边界上，Python 不使用隐藏答案校验，只检查输出是否为合法 JSON list、是否以输入数字开头、是否以 1 结尾、是否存在明显格式错误。对于格式失败样本，重新调用模型按同一规则输出。该任务线上表现接近满分，后续没有继续占用提交机会。

3.4 Task4: CoNaLa string concatenation

Task4 是字符串拼接任务。错误主要来自引号边界、空格保留和转义字符处理。方案将提示固定为“抽取每个待拼接字符串，再按给定顺序拼接”，输出时不加引号、不解释。后处理仅清理模型可能添加的外围引号或 Markdown 标记，不改写字符串内容。该任务同样已接近满分。

3.5 Task5: tweet sadness detection

Task5 是二分类情绪标注，输出 Sad 或 Not sad。该任务的关键是标签标准一致性：有些 tweet 包含负面词但不是悲伤，有些包含讽刺或隐含失落。我们使用少量代表性 examples 固定标签含义，并采用低温度输出和严格标签抽取。对于不符合 Sad/Not sad 的输出，只做格式重试，不通过程序重判情绪。

3.6 Task6: MNLI same genre classification

Task6 是自然语言推断风格的二分类任务，输出 Y 或 N。我们比较了 strict prompt 和 JSON prompt 两类复核方式。公开验证中 JSON 复核准确率略高，稳定样本准确率较高；但最后一轮 jsonY\_extra 分支在官方平台回撤，说明过度扩张 N -> Y 会引入假阳性。

最终方案只继承已验证的稳定分支，不采用回撤的二次扩张包。报告中的最终预测包以 81.72 基线为准，没有叠加 task6\_jsonY\_extra 的回撤改动。

3.7 Task7: Jeopardy answer generation

Task7 是本方案后期重点突破点。早期我们尝试过 zero-shot、BM25 ICL、high-k ICL、多温度采样、多候选裁决和长上下文增强，整体收益都不稳定。主要原因是 Task7 涉及大量长尾实体知识，Qwen3-4B 经常给出同槽位错误实体，例如演员、城市、作品名或政治人物。

最后发现一个更可控的错误类型：模型会把题面中出现的相关词直接当答案复制出来。例如：

| 当前预测           | 题面线索                                                                      | 复核候选              |
|----------------|---------------------------------------------------------------------------|-------------------|
| peru           | Once known as Upper Peru                                                  | bolivia           |
| gambia         | The Gambia is the smallest independent mainland country on this continent | africa            |
| utilitarianism | Father of Utilitarianism                                                  | jeremy bentham    |
| knickerbocker  | Diedrich Knickerbocker, a “sleepy” novelist                               | washington irving |

因此我们设计了 Task7 copy-consensus 复核：

- 1. 触发条件：当前预测完整出现在 category 或 clue 中；
- 2. 四路模型复核：zero、anticopy、slotjson、knowledge；
- 3. 门控条件：候选不能再次被题面完整包含，不能超过 5 个词，不能只是原复制短语的扩写；
- 4. 一致性条件：至少 3/4 通道一致，或 zero 与 knowledge 一致；
- 5. 窄模式补充：对 one of these 和 translated from 类题，允许 slotjson 与 knowledge 一致时替换。

该分支不是人工按 test\_sample\_id 写答案，而是由 Qwen3-4B 在多个提示通道下独立复核，Python 只做一致性筛选。线上对比显示，在一个相邻基线上，Task7 copy-consensus plus2\_v3 带来约 +0.23 总分收益。但由于该分支没有在 81.72 主基线上完成官方评测，本次开源默认方案不启用该分支，仅将其作为可复现实验分支保留。

### 3.8 Task8: kernel generation

Task8 是代码生成任务，评分不仅看代码能否调用，还看计算结果是否正确。早期只修 runtime/API 错误时，收益很快变小。后续方案改为执行反馈链路：

```
Qwen3-4B 生成代码
-> wrapper 签名解析
-> 公开语义 reference 构造
-> 本地/官服执行 smoke test
-> 失败类型归因
-> 通用 wrapper 修复规则
-> 重新打包提交
```

修复规则不以隐藏答案为依据，而以公开函数签名和 wrapper 描述为触发键。例如：

| 错误类型                                           | 通用修复方式                                               |
|------------------------------------------------|------------------------------------------------------|
| 不存在的 in-place 顶层 API，如 <code>torch.pow_</code> | 改为 <code>torch.pow</code> 并处理 <code>out.copy_</code> |
| <code>linalg.xxx</code> namespace 缺失           | 构造轻量 namespace wrapper                               |
| <code>out</code> 参数返回错误                        | 固定 <code>out.copy_(result); return out</code>        |
| fused API 语义不一致                                | 用 PyTorch 等价操作表达公开语义                                 |

Task8 当前方案采用保守的 Triton/PyTorch hybrid：优先保留模型生成的可执行代码；对 API 和 wrapper 层面错误，使用通用规则修复；对无法构造可靠 reference 的样本不做激进替换。

## 4. 实验与结果

### 4.1 官方提交成绩演进

| 阶段                       | 提交包                                                                                     | 官方分数  | 说明                               |
|--------------------------|-----------------------------------------------------------------------------------------|-------|----------------------------------|
| 可复现合规版                   | <code>final_reproducible_20260519.zip</code>                                            | 67.57 | 去除高风险硬编码后的保守可复现版本                |
| Task2/Task8 强化基线         | <code>final_submit_20260520_task2_structured_two_model_or_verbs_on_8012.zip</code>      | 80.97 | 进入 80 分段                         |
| Task6 稳定重判 + Task8 full7 | <code>final_submit_20260520_task6_stable_n_to_y_plus_task8_full7_on_8097.zip</code>     | 81.60 | 稳定提升                             |
| Task2 noun 高置信重判         | <code>final_submit_20260520_task2_noun_highconf_on_8160.zip</code>                      | 81.72 | 当前已验证最好基线                        |
| Task6 二次扩张               | <code>final_submit_20260520_task6_jsonY_extra_on_8172.zip</code>                        | 81.47 | 回撤，最终不采用                         |
| Task7 copy-consensus 对比包 | <code>final_submit_20260520_task7_copy_consensus_plus2_v3_on_task6jsonY_8172.zip</code> | 81.70 | 相对同基线提升约 +0.23；作为实验分支保留，未纳入最终默认包 |

### 4.2 最终包格式校验

最终预测包：

```
submissions/final_submit_20260520_task2_noun_highconf_on_8160.zip
```

本地校验结果：

| 文件                               | 行数  |
|----------------------------------|-----|
| <code>openseek-1-v1.jsonl</code> | 500 |
| <code>openseek-2-v1.jsonl</code> | 500 |
| <code>openseek-3-v1.jsonl</code> | 500 |
| <code>openseek-4-v1.jsonl</code> | 500 |
| <code>openseek-5-v1.jsonl</code> | 500 |
| <code>openseek-6-v1.jsonl</code> | 500 |
| <code>openseek-7-v1.jsonl</code> | 500 |
| <code>openseek-8-v1.jsonl</code> | 166 |

全部文件直接位于 zip 根目录，字段为 `test_sample_id` 和 `prediction`，无空预测。

## 5. 合规性说明

### 5.1 未使用外部数据

本方案只使用 OpenSeek 官方数据目录中的 `Definition`、`examples` 和 `test_samples`。没有使用外部知识库、搜索引擎、人工标注测试集答案或自建训练数据。

### 5.2 后处理边界

Python 后处理的边界如下：

| 允许内容  | 本方案用途                          |
|-------|--------------------------------|
| 格式抽取  | 从 JSON、代码块或文本中提取最终 prediction  |
| 一致性比较 | 比较多个 Qwen3-4B 通道是否给出同一候选       |
| 执行检查  | 对 Task8 公开 API 语义构造 smoke test |
| 打包校验  | 检查 jsonl 文件名、行数、字段和空值          |

不允许内容包括：根据隐藏答案改写预测、使用外部知识直接填答案、将 `test_sample_id -> answer` 作为最终规则。本方案的 Task7 和 Task2 修改均来自 Qwen3-4B 复核输出及通用门控，Task8 修复以公开 wrapper 签名和函数语义为触发条件。

### 5.3 FlagScale 复现路径

官方仓库 README 给出了 FlagScale 启动方式。本方案保存了对应复现脚本：

```
bash final_reproducible/start_official_flagscale_service.sh
```

官服验证记录包括：

- `/v1/models` 能返回 Qwen3-4B；
- `max_model_len` 为 `40960`；
- Task1 的 30K Round 1 prompt 已通过官服 FlagScale 服务真实调用，返回 `prompt_tokens=30206`。

## 6. 复现说明

### 6.1 环境准备

按官方 README 下载 Qwen3-4B:

```
hf download Qwen/Qwen3-4B --local-dir Qwen3-4B
# 或
modelscope download --model Qwen/Qwen3-4B
```

配置长上下文 rope scaling 后, 启动 FlagScale:

```
cd FlagScale
python run.py --config-path .. --config-name llm_config action=run
```

### 6.2 长上下文证据链生成

快速生成每任务 3 条证据:

```
python3 final_reproducible/build_long_context_evidence.py --sample-limit 3 --write-prompts
```

全量生成证据:

```
python3 final_reproducible/build_long_context_evidence.py --all --write-prompts
```

输出包括:

```
final_reproducible/long_context_evidence/long_context_evidence.jsonl
final_reproducible/long_context_evidence/long_context_evidence_summary.json
final_reproducible/long_context_evidence/prompts/
```

### 6.3 提交包校验

```
python3 final_reproducible/validate_submission.py \
submissions/final_submit_20260520_task2_noun_highconf_on_8160.zip
```

校验内容包括文件名、行数、字段集合、重复 ID 和空 prediction。

## 7. 局限与后续工作

第一, Task7 仍受 Qwen3-4B 长尾知识限制。copy-consensus 修复的是“抄题面词”这一类结构性错误, 不能解决所有 Jeopardy 长尾实体问题。若继续优化, 需要更可靠的候选排序信号, 但不能引入外部知识库。

第二, Task8 仍有执行正确性空间。当前方案已经从“能运行”推进到“部分语义正确”, 但还有一些 wrapper 缺少可靠 reference, 继续扩展公开语义测试可以带来增益。

第三, Task2 的词性边界仍可细化。当前 noun 高置信分支带来了正收益, 但如果继续扩张, 容易出现过拟合或回撤。因此最终只采用高置信范围。

## 8. 结论

本方案针对长上下文 ICL 自动标注任务, 形成了一个可复现、可解释的工程流程: 用 30K/16K 长上下文规则理解轮组织官方 examples 和任务边界, 用任务类型分层和结构化提示降低模型漂移, 用多通道 Qwen3-4B 复核处理低分任务, 用执行反馈链路



处理代码生成任务。最终开源默认预测包对应官方平台已验证的 **81.72** 最高分版本；未在该基线上完成官方评测的 Task7 copy-consensus 分支仅作为可选实验保留。

## 参考资料

1. FlagOS 赛事页面: <https://flagos.io/RaceDetail?id=296fmsd8&lang=cn>
2. OpenSeek LongContext-ICL-Annotation 官方仓库: <https://github.com/FlagAI-Open/OpenSeek/tree/main/openseek/competition/LongContext-ICL-Annotation>
3. FlagOS 官方文档: <https://docs.flagos.io/en/latest/overview.html>
4. FlagScale 官方仓库: <https://github.com/FlagOpen/FlagScale>
5. Qwen3-4B 模型页面: <https://huggingface.co/Qwen/Qwen3-4B>